

Recurrent Neural Networks and Genetic Algorithms

Video Lesson

Duration: 00:17:56

Slide Deck

M12S07 (<https://canvas.tamu.edu/courses/430127/files/92570700?wrap=1>) 

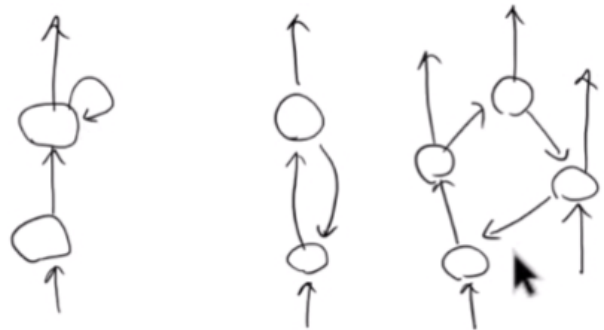
Lesson Transcript

In this final lecture of the machine learning module, we will look at recurrent neural networks and genetic algorithms.

Recurrent Networks

Connection graph can contain cycles, e.g. reciprocal connections: i.e. not strictly feed-forward.

- Statistical mechanics based models (associative or content-addressable memory): Hopfield network, Boltzmann machines, etc.
- Sequence encoding: Simple Recurrent Network, etc.
- Other biologically motivated networks: laterally connected self-organizing maps, etc.

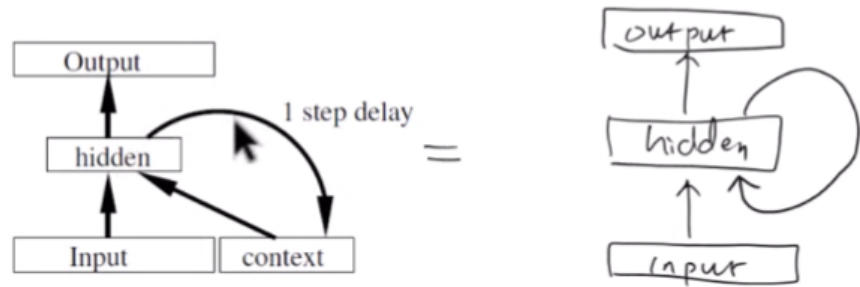


Recurrent neural networks are networks where the connection graph contains cycles. For example, it could be reciprocal connections like this, or a loop to itself, or some multi-hop loop coming back to itself.

In other words, it is not strictly feed-forward. Recurrent neural networks include statistical mechanics-based models such as the Hopfield network or the Boltzmann machines, and these are used for associative or content-addressable memory. The more common use nowadays is something based on the simple recurrent network and is heavily used for sequence encoding, for example, for natural language or speech processing.

There are other biologically motivated neural networks that have this kind of connection or that kind of connection, and some of these include laterally connected self-organizing maps.

Simple Recurrent Network (Elman Network)



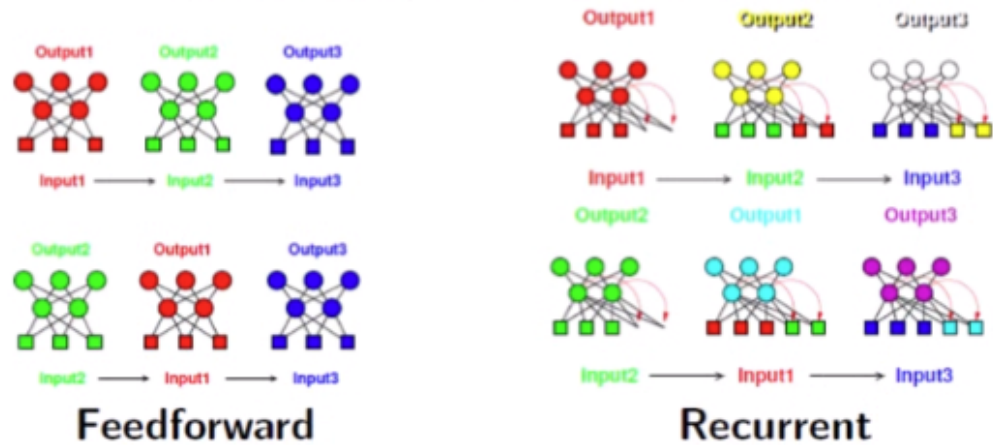
- Sequence encoding.
- Hidden layer activation from previous time step used as input.
- Use standard back-propagation learning.

The simplest form of recurrent neural network could be the simple recurrent network, otherwise known as the Elman network. Here we can see that there is basically a feed-forward neural network, but the hidden layer activation is copied over to this context vector with a one-step delay. So, the activation of the hidden layer will be used together with the next input that comes in, hence it will form this loop.

So, if we look at the mechanism, we can also represent the same thing as shown on the right-hand side. Here, all of these units in the hidden layer loop back and stimulate themselves. This kind of network can be used for sequence encoding. As we can see, the hidden layer activation from the previous time step is used as input.

Using the back-propagation learning algorithm, we can train these neural networks.

Recurrent Neural Networks



- Feedforward networks:
 - No memory of past inputs. Output only based on the present input.
- Recurrent networks:
 - Past inputs affect the present output.

The most interesting feature of the recurrent neural network is that its output will not only be based on the immediate input but also it will be based on the sequence of input that came in prior to the current input. For comparison, let us look at the feedforward network and how the output is generated given a sequence of inputs, like Input 1, Input 2, and Input 3.

At any moment, a feedforward network will just generate an output directly based on the immediate input. When Input 1 is given, the Output 1 will only be based on Input 1. When Input 2 is given, again, the output will only be based on Output 2. And when Input 3 is given, the output will only be based on this particular input.

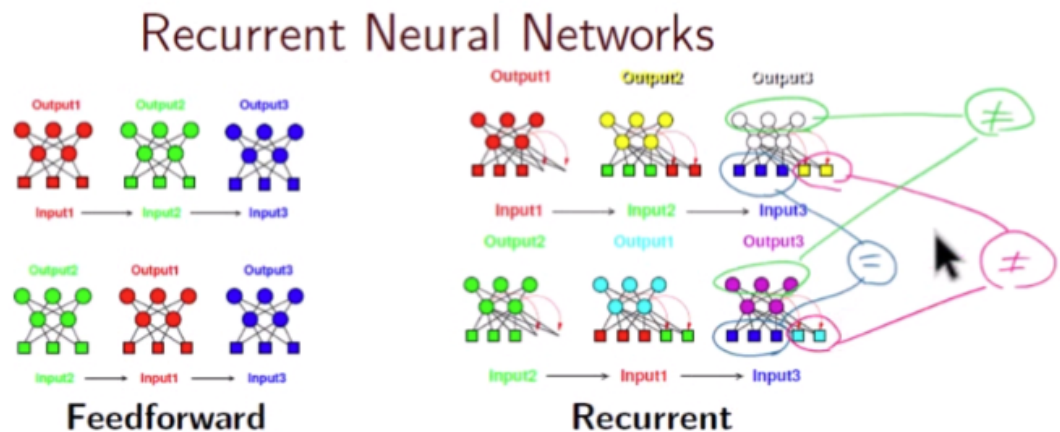
The ordering does not matter. So, for these two cases, when Input 3 is given, regardless of the ordering of the previous input, Input 1, Input 2, or Input 2, Input 1, the output will always be the same. So, in this sense, we can say that feedforward neural networks have no memory of past inputs, and the output is only based on the present input.

Now let us consider recurrent neural networks. And let us use the same input sequence. Input 1, input 2, Input 3 in the top case. Input 2, Input 1, Input 3 in the bottom case. In this case, Input 1 comes in, and Output 1 is generated as usual. But the hidden layer activity is looped back, and it is used as part of the input for the second time step.

So, the second time step response will be based on Input 2 and the remembered activity from the previous step. So, this output would be very different from that because we have extra information coming in. Finally, when we go to Step 3, this hidden unit activity, which is based on the immediate

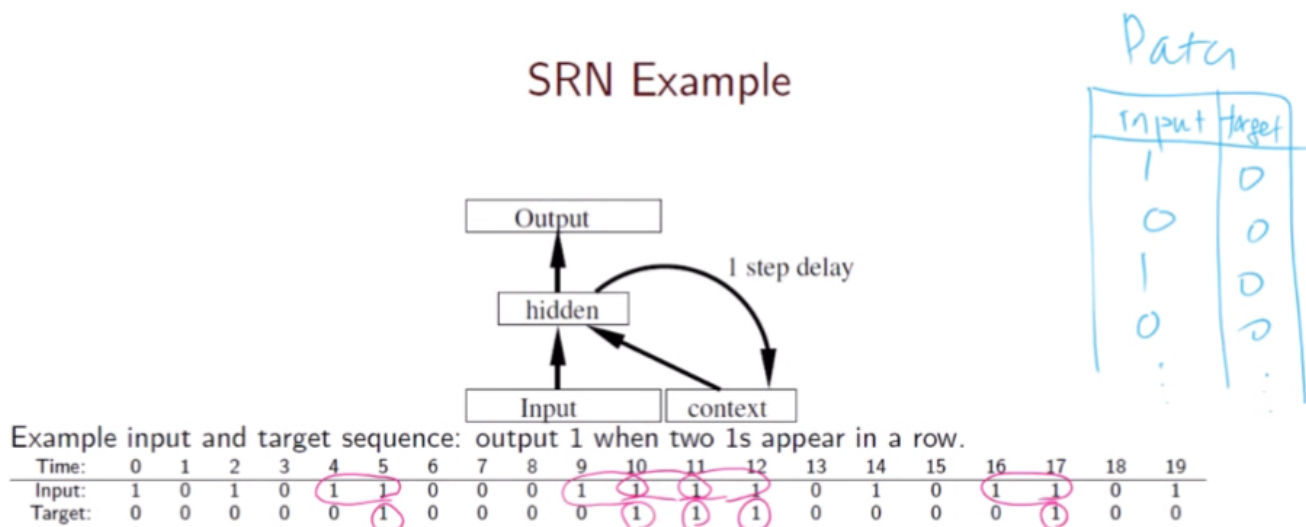
input, Input 2, and remembered input from Input 1, would go in and form the context input alongside Input 3, which is the immediate input, and we will get the output.

When the input order is changed to Input 2, Input 1, what happens is that this hidden layer activity will be different from that hidden layer activity. And because that changed hidden layer activity is used as the context input, when it is combined with the immediate Input 3, it will generate yet a different output from the first case.



- Feedforward networks:
 - No memory of past inputs. Output only based on the present input.
- Recurrent networks:
 - Past inputs affect the present output.

We can summarize this as shown here. Although Input 3 is the same, since the context input, which is based on the past sequence of inputs, is different, the output is also different.



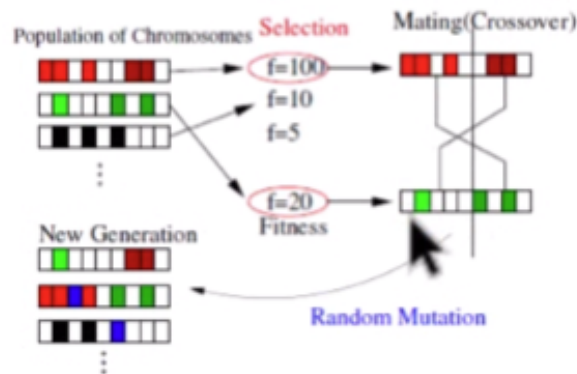
Here is a simple example that shows how this recurrent network can learn to find a certain pattern in the input sequence.

The sequence shown here is basically an input dataset that looks like this. So, the input is 1, and the target is 0. Input 0, target 0. Input 1, target 0. They correspond to each column here.

Here, if we look at the target value, what we can see is that when two subsequent input is 1, then the target value is 1. As we can see, whenever there is a back-to-back 1 in the input sequence, the target is 1. Again, back to back 1, inputs, input, then target value of 1. Again, back-to-back input sequence, then the target is 1, and so on.

Of course, this is the simplest example. We can imagine how this kind of data set could be constructed based on some more complex tasks, like word prediction or sequence recognition. We will look at the recurrent neural networks in more detail when we go to the deep learning module.

Genetic Algorithms



Evolution as a problem solving strategy:

- population of solutions, where each chromosome represent an individual
- selection based on fitness function: survival of the fittest
- mating (cross-over) and reproduction
- random mutation

Next, let us consider genetic algorithms. In genetic algorithms, we want to use evolution as a problem-solving strategy. So, we have a population of solutions where each chromosome, as shown here, represents an individual solution.

Each of these may have a variable performance, and we can measure the performance using a

fitness function, f . The next step is to select which ones to preserve and mate and carry over to the next generation. This is called selection. We can summarize this as the survival of the fittest. Once we pick these good ones or high-performance ones, we combine them.

For example, let us say here is the red chromosome, and here is the green chromosome. Then we align them and draw a crossover line here and produce a new solution or the offspring. When we have these two parent chromosomes, then we end up with two offspring. Also, there is a random mutation. That is done with a certain probability. We can see that these three red genes turned on, and two green turned on here correspond to that offspring.

Then, when we cross over this part of the second parent and this part of the first parent, we get this one. In this case, there is no mutation, so we can see that the sequence is exactly the same. Sometimes, whole chromosomes may be carried over if they have high, high fitness. Also, we may or may not mutate some of these genes.

Properties of Genetic Algorithms

- each chromosome encodes a solution
- similar to hill-climbing search
- parallel search
- works for both immediate or delayed reward

In genetic algorithms, each chromosome encodes a solution. Because we are choosing the high-fitness chromosomes, the algorithm is similar to a hill-climbing search. Also, we can conduct this in parallel quite easily. Finally, it works both for immediate reward, as in supervised learning, or delayed reward, as in reinforcement learning.

Designing a GA Solution to a Problem

There are many different issues:

- What is the fitness function?
- How is an individual chromosome represented (how to encode) and what does it represent?
- How are individuals selected?
- How do individuals reproduce?

There are several different things to consider when designing a genetic algorithm solution to a particular problem. There are several issues.

What should be the fitness function?

How is an individual chromosome represented, that is, what is the encoding, and what does it represent?

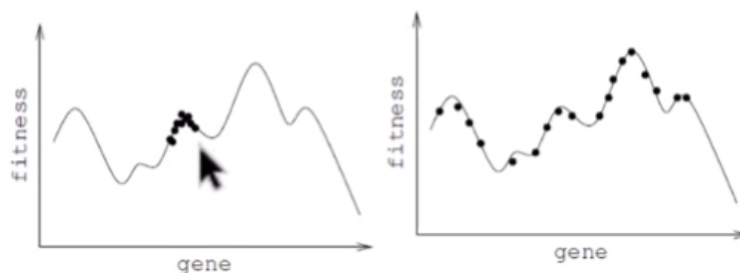
How are the individuals selected?

Should we just line them up and select the top 10% or choose it probabilistically based on the fitness values and so on?

How do these individuals reproduce?

Who is allowed to reproduce, and how to crossover?

Issues in GA: Diversity



How to maintain diversity:

- Letting only successful ones to reproduce can seriously reduce the gene pool and an epidemic can wipe out the whole population: solution can not generalize in new and unexpected conditions.
- Converged population can often times be found at local minima, not at the global optimum.

One of the main issues with genetic algorithms is diversity, especially if we use some kind of elitist

selection strategy.

For example, just select the top 10%. Then usually, what we end up with is a crowding near a local maximum.

Letting only successful ones reproduce can seriously reduce the gene pool, and something bad can wipe out the entire population. Also, solutions cannot be generalized in new and unexpected conditions.

As I mentioned, they tend to crowd near local maxima. With diversity, we can possibly find individuals that reach the global maximum.

More Issues in GA

- Cross-over strategy: success depends on how genes are encoded (or represented).
- Not too much theoretical understanding about why it works so well.
- Crevices in fitness landscape: similar to spikes in hill climbing.
- How to combine learning with evolution.
- How to use cultural leverage.

There are several more issues to consider. For example, what kind of crossover strategy can we use? The success really depends on how the genes are encoded or represented.

If we cut at a strange location, then some functions can be broken. There is not too much theoretical understanding about why it works so well. There could be sharp peaks or crevices in the fitness landscape, and it will be difficult to either get out of this crevice or climb up to the high peak.

More issues include how to combine learning with evolution and, relating to that, how to leverage culture.

GA as a Learning Algorithm



- An individual chromosome may not seem to learn, but when we look at the evolution of individuals over time, they can be seen as adapting, and thus learning to cope with the environment.
- If each individual encodes a function rather than a simple solution, the above point becomes clearer. At each generation, the parameters in the function can be seen as being adapted.
- Fitness can then be measured by using the function with the given parameters in specific tasks.

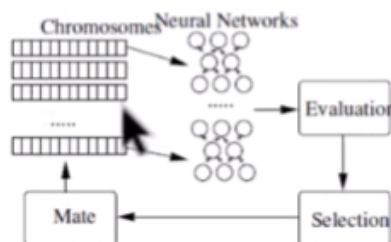
If we look at the single chromosome, all the values are fixed, and they are not modified within that particular instance. We can say that an individual chromosome may not seem to learn. But when we look at the evolution process as a whole, and if we look at these individuals over time, we can see them adapting.

Thus, learning to cope with the environment is what we can observe.

We can consider that each individual includes a function rather than a simple solution. In this case, we can think of the function by changing its parameters over the generations as being adapted.

Fitness can then be measured by using this function with the given parameters in a specific task.

GA as a Learning Algorithm: Neuroevolution



Evolving neural networks:

- Chromosomes encode neural networks (connection topology and connection weights).
- Evaluate, select, and reproduce new population of neural networks.

Problem: individual neurons performing good work may get lost.

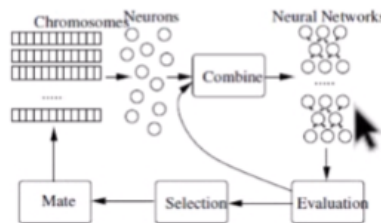
Genetic algorithms are a really good match with neural networks. A single chromosome can fully specify the connection weights of a fixed topology in a neural network. Let us see if we have 100

chromosomes. Then we can instantiate them as 100 neural networks.

Then we put these neural networks in the task and evaluate and select, mutate, mate to, to generate the next generation.

The problem with this is individual neurons that are doing well can get lost due to crossover. For example, if this crossover line was drawn between these two weights, then this neuron might lose its function, which required these two connections to have a certain value.

Neuroevolution: Evolving Individual Neurons



SANE: Moriarty and Miikkulainen

- Chromosomes encode individual neurons.
- Neurons solve sub-problems and the ones that solve the problem well gets a chance to participate in a network in the next generation.
- Better diversity is maintained.

One way to overcome that issue is to evolve individual neurons rather than a whole network.

In this approach by Moriarty and Miikkulainen, each chromosome would encode a single neuron. These neurons are combined to form a neural network, and the fitness for each of these neurons is computed based on the fitness of the particular neural network that they are participating in. With this strategy, neurons learn to solve sub-problems, and the ones that are involved in a neural network that solves the problem well, get the chance to participate in a network in the next generation.

It was empirically shown that this kind of approach would result in better diversity.

GA Demo

Neuroevolution: <http://www.cs.utexas.edu/users/nn/>

- Generation of melodies (Chen and Miikkulainen)
- Gaming AI; harvesters and predators (Stanley and Miikkulainen)
- Non-markovian control (Gomez and Miikkulainen)

There are several interesting demos on genetic algorithms applied to neural networks. For example, this URL or the demo by Ken Stanley.

[Video is shown]

Here is an example by Ken Stanley. The algorithm is called NEAT, Neural Evolution of Augmenting Topologies. Here we can see two agents, the yellow one and the red one, and they have two range sensors. One is to detect food, and the outer ring is the range sensor to detect our enemy's position. We have four sensors each. Thus, we have eight total range sensors.

The next one is the energy level sensor. This is needed because we can actually gobble up our enemy if we have a higher energy than our enemy. The next one is the boundary sensor. We cannot go beyond the boundary. The last one is a fixed bias input, and they always have the same value. Now we have two output units to control the rotation and thrust.

These are some evolved neural networks. The top one corresponds to the yellow agent, and the bottom one corresponds to the red agent. Let us look at an interesting, evolved network much more complex than this.

[Video is shown]

We can see the yellow guy and the red guy moving around, eating food to increase their energy. And when they move around, they will consume energy, so they do not want to move too much. Some interesting behavior, the yellow one is kind of taunting the red one. Right now, it looks pretty bad for the red one.

Let us see what happens.

[Video is shown]

It is interesting that the red one was waiting before it gobbled up the food. By doing that, it increased its energy enough to be able to defeat its enemy. So, this is the neural network topology for the yellow one, and this is the neural network topology for the red one. As we can see, the red one, which was victorious, has a much more complex neural network controller.

[<https://www.cs.ucf.edu/~kstanley/>] ⇨ [<https://www.cs.ucf.edu/~kstanley/>)]

I encourage you to look at these many interesting demos by Ken Stanley. You can go to his webpage and look at some of these demos.

[<https://www.cs.ucf.edu/~kstanley/demos.html>] ⇨ [<https://www.cs.ucf.edu/~kstanley/demos.html>]
]

Here is Ken's demo page. You can scroll down, and this is the demo that we saw and some more interesting stuff.

GA Demo

Neuroevolution: <http://www.cs.utexas.edu/users/nn/>

- Generation of melodies (Chen and Miikkulainen)
- Gaming AI; harvesters and predators (Stanley and Miikkulainen)
- Non-markovian control (Gomez and Miikkulainen)

This is the end of this lecture, and this is the last of the lecture for this module, the machine learning module. Next, we will move on to deep learning.